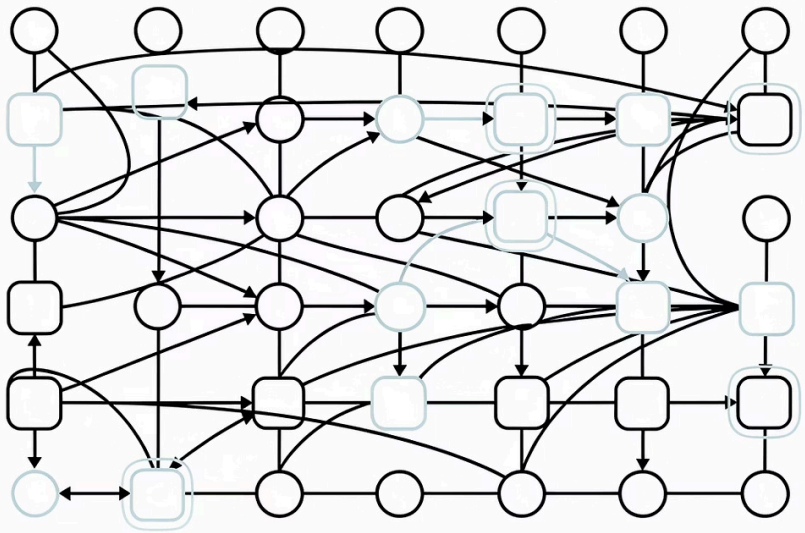


Backpropagation: The Engine of Deep Learning

A comprehensive guide to understanding how neural networks learn — from intuition and mathematics to PyTorch implementation. We cover forward and backward passes, gradient propagation, loss functions, optimizers, and the full training loop.

MACHINE LEARNING | PYTORCH | DEEP LEARNING

Agenda

01

Forward Pass

How data flows through the network, layer by layer.

02

Backward Pass & Gradients

Chain rule, gradient propagation, dL/dw, dL/dx.

03

Loss Functions

MSE, Binary Cross-Entropy, Cross-Entropy.

04

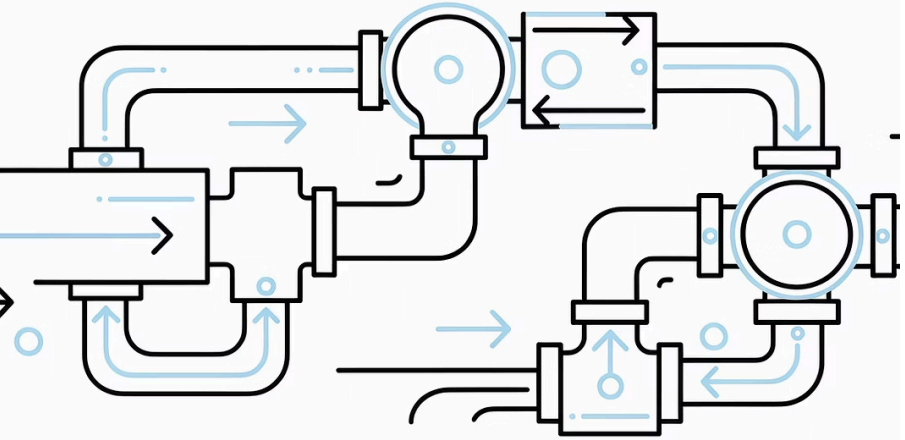
Optimizers

SGD, Momentum, AdaGrad, RMSProp, AdaDelta.

05

Training Loop

Learning rate, epochs, regularization, dropout, autograd, tensors, batches.



Forward Pass: Data Flowing Through the Network

The **forward pass** is the process of transforming an input **x** into a prediction **ŷ** by sequentially applying learnable transformations at each layer. For a feedforward network with one hidden layer:

Layer Computations

Hidden layer: $z^1 = W^1x + b^1$

Activation: $a^1 = \sigma(z^1)$

Output layer: $z^2 = W^2a^1 + b^2$

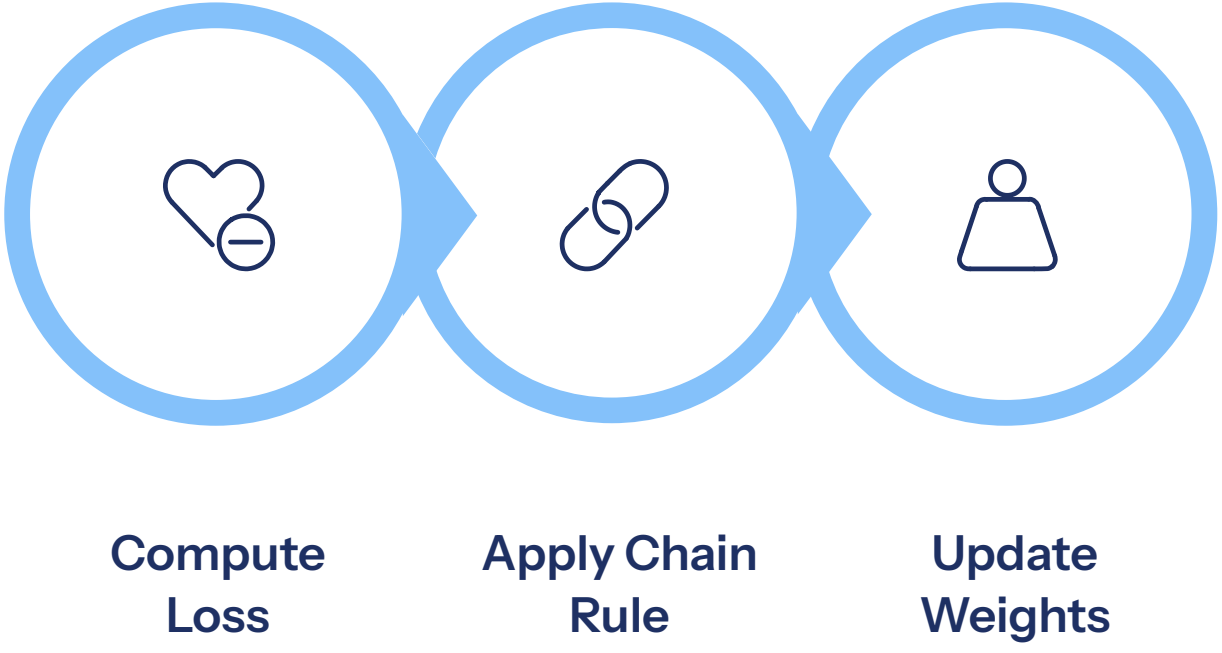
Prediction: $\hat{y} = \text{softmax}(z^2)$

Key Concepts

- Each layer applies a linear transformation followed by a non-linear activation (ReLU, sigmoid, tanh).
- All intermediate values (z^1, a^1, z^2) are **cached** during the forward pass — they are required for computing gradients in the backward pass.
- The forward pass produces the loss value L, which measures prediction error.

Backward Pass: Learning from Mistakes

The **backward pass** (backpropagation) computes how much each weight contributed to the error, using the **chain rule of calculus**. Gradients flow from the output back to the input.



Every weight in the network receives a gradient signal telling it: "if you increase slightly, does the loss go up or down, and by how much?" This is the fundamental mechanism that enables learning.

Gradient Propagation: The Chain Rule in Action

Backpropagation relies entirely on the **chain rule**: the derivative of a composed function is the product of derivatives along the path. For a computation $y = f(x)$ and loss $L = g(y)$:

Key Gradient Formulas

dL/dy — How the loss changes with respect to the layer output y. Computed first (closest to the loss).

dL/dw — How the loss changes with respect to weight W. Used to update W. Formula: $dL/dW = dL/dy \cdot x^T$

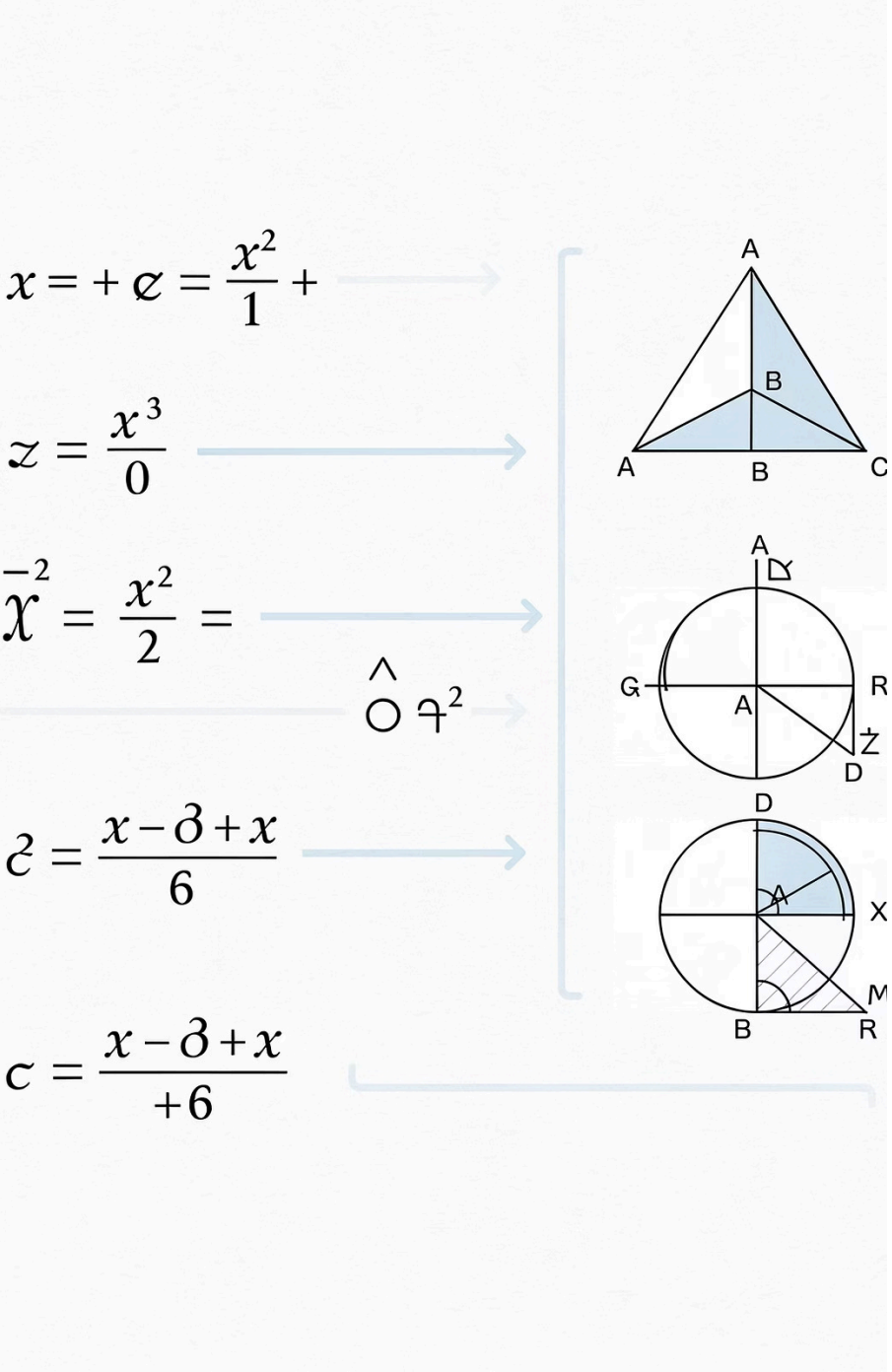
dL/dx — How the loss changes with respect to input x. Passed further back. Formula: $dL/dx = W^T \cdot dL/dy$

Chain Rule Decomposition

For a two-layer network:

$$dL/dW^1 = dL/dz^2 \cdot dz^2/da^1 \cdot da^1/dz^1 \cdot dz^1/dW^1$$

Each term is computed locally at each node — this is what makes backpropagation efficient and modular. No global information is needed.



Deriving Backprop Formulas: One Hidden Layer Network

Consider a network: **x → [W^1, b^1] → ReLU → [W^2, b^2] → ŷ → L**. Here is the full symbolic derivation of all gradients for each parameter.

Output Layer Gradients

$\delta^2 = dL/dz^2$ (depends on loss + output activation)

$dL/dW^2 = \delta^2 \cdot (a^1)^T$ | $dL/db^2 = \delta^2$

Backpropagating Through ReLU

$dL/da^1 = (W^2)^T \cdot \delta^2$

$\delta^1 = dL/da^1 \odot ReLU'(z^1)$ where $ReLU'(z) = 1$ if $z > 0$, else 0

Hidden Layer Gradients

$dL/dW^1 = \delta^1 \cdot x^T$ | $dL/db^1 = \delta^1$

These gradients are then used by the optimizer to update W^1 and b^1.

Loss Functions: Measuring Prediction Error

The loss function quantifies how far the network's predictions are from the ground truth. Choosing the right loss function is critical — it shapes the gradients and drives learning.

MSE (Regression)

$L = (1/n) \Sigma (y_i - \hat{y}_i)^2$

Used for continuous targets. Penalizes large errors quadratically. Gradient: $dL/d\hat{y} = -2(y - \hat{y})/n$

Binary Cross-Entropy

$L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$

Used for binary classification with sigmoid output. Heavily penalizes confident wrong predictions.

Cross-Entropy

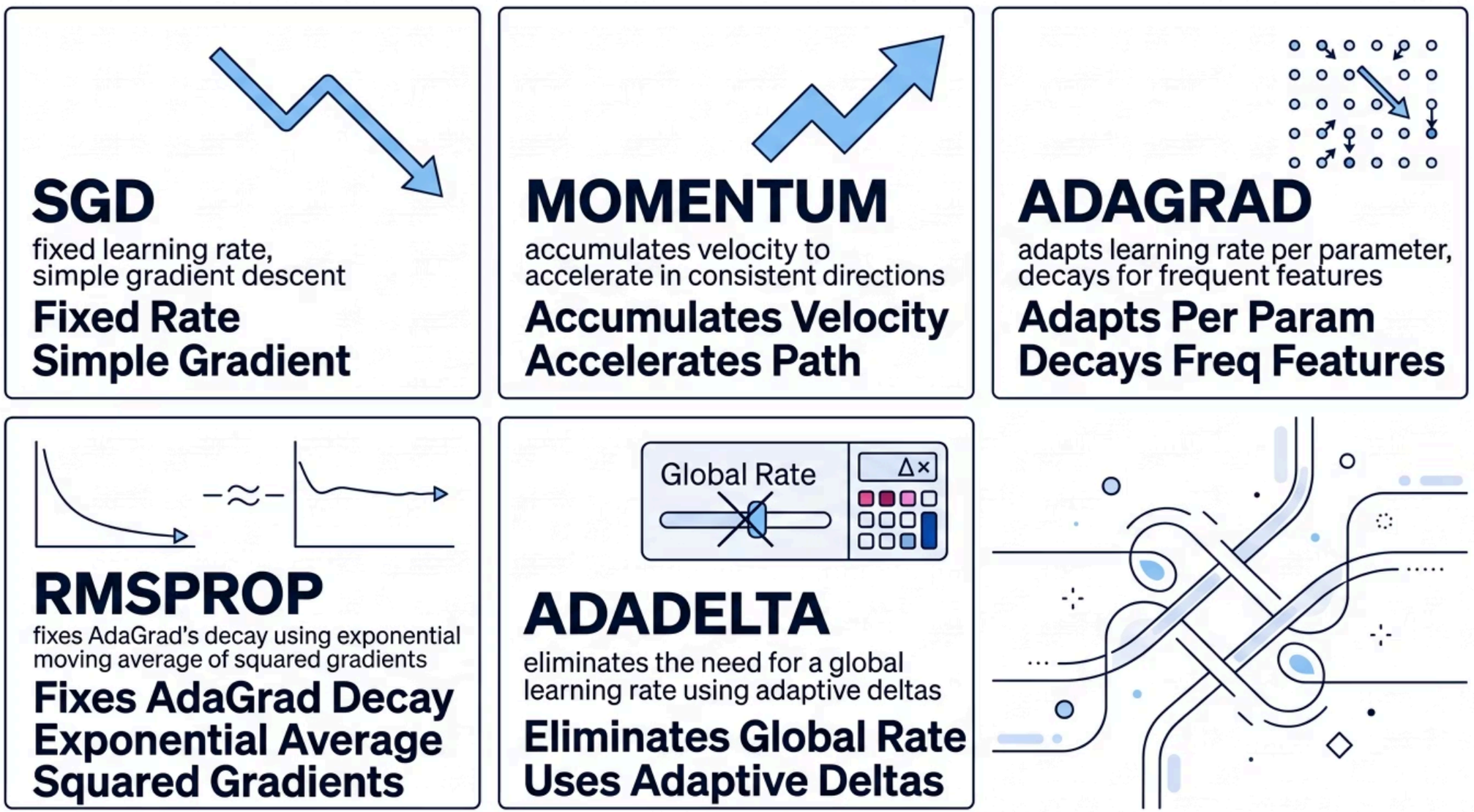
$L = -\Sigma\, y_i \cdot \log(\hat{y}_i)$

Used for multi-class classification with softmax. Minimizes negative log-likelihood of the correct class.

In PyTorch: `nn.MSELoss()`, `nn.BCEWithLogitsLoss()`, `nn.CrossEntropyLoss()` — the latter combines softmax + NLL for numerical stability.

Optimizers: How Weights Are Updated

Optimizers define the strategy for updating weights using computed gradients. They all follow the basic rule $W \leftarrow W - \alpha \cdot \nabla W$, but differ in how they adapt the learning step.



SGD and Momentum

Stochastic Gradient Descent (SGD)

The simplest optimizer. Updates weights using the gradient of the current mini-batch:

$$W \leftarrow W - \alpha \cdot \nabla W$$

Fast but noisy — each mini-batch gradient is a noisy estimate of the true gradient. Can oscillate in ravines and get stuck in sharp local minima. PyTorch: `torch.optim.SGD(lr=0.01)`

SGD with Momentum

Adds a **velocity term** that accumulates past gradients:

$$v \leftarrow \beta \cdot v + \nabla W$$

$$W \leftarrow W - \alpha \cdot v$$

β is typically 0.9. Momentum dampens oscillations and accelerates convergence along consistent gradient directions — like a ball rolling downhill, gaining speed. PyTorch: `SGD(momentum=0.9)`

AdaGrad, RMSProp & AdaDelta

AdaGrad

Maintains a sum of squared gradients G for each parameter. Divides the learning rate by $\sqrt{G}$: $\mathbf{W \leftarrow W - (\alpha/\sqrt{G}) \cdot \nabla W}$. Great for sparse features; problem: G grows forever, making learning rate vanish.

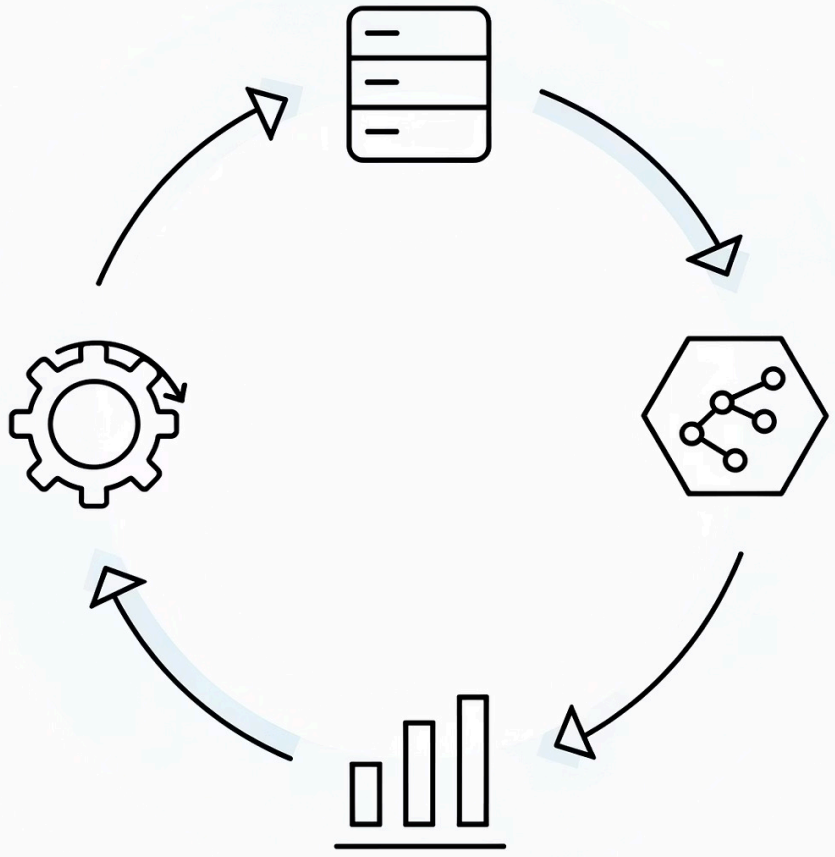
RMSProp

Fixes AdaGrad by using an **exponential moving average** of squared gradients: $\mathbf{G \leftarrow \rho \cdot G + (1-\rho) \cdot \nabla W^2}$. The learning rate remains effective throughout training. PyTorch: `torch.optim.RMSprop()`

AdaDelta

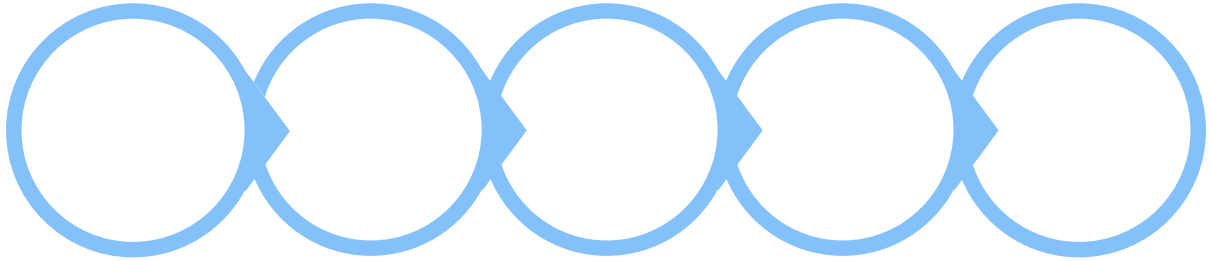
Extends RMSProp by also tracking the exponential average of squared parameter updates. **Eliminates the need for a global learning rate** entirely — making it more robust and self-tuning. PyTorch: `torch.optim.Adadelta()`

In practice, **Adam** (Adaptive Moment Estimation) combines Momentum + RMSProp and is the most widely used optimizer: `torch.optim.Adam(lr=1e-3)`



The Training Loop

The training loop is the core procedure that brings all components together — data, model, loss, and optimizer — into a repeating cycle of learning.



In PyTorch, this loop is written explicitly, giving the practitioner full control over each step. This transparency makes debugging and experimentation straightforward.

Training Loop in PyTorch

```
for epoch in range(num_epochs):
    for X_batch, y_batch in dataloader:

        # 1. Forward pass
        y_pred = model(X_batch)

        # 2. Compute loss
        loss = criterion(y_pred, y_batch)

        # 3. Zero gradients (IMPORTANT!)
        optimizer.zero_grad()

        # 4. Backward pass
        loss.backward()

        # 5. Update weights
        optimizer.step()

    print(f"Epoch {epoch}, Loss: {loss.item():.4f}")
```

⚠ **Common mistake:** Forgetting `optimizer.zero_grad()` causes gradients to **accumulate** across batches, leading to incorrect updates and unstable training.

Learning Rate: The Most Important Hyperparameter

What It Controls

The learning rate α determines **how large a step** the optimizer takes in the direction of the negative gradient. It scales every weight update:

$$W \leftarrow W - \alpha \cdot \nabla W$$

Too large → training diverges or oscillates. Too small → training is slow and may get stuck. Finding the right α is critical.

Strategies

- **Fixed LR:** Simple, risky for large problems.
- **LR Scheduling:** Reduce α on plateau or by epoch. `torch.optim.lr_scheduler`
- **LR Warmup:** Start small, ramp up, then decay — common in Transformers.
- **Cyclical LR:** Oscillate between min and max bounds — helps escape local minima.

Number of Epochs & Overfitting

An **epoch** is one complete pass through the entire training dataset. Training for the right number of epochs is a balance between underfitting and overfitting.

Too Few Epochs

Underfitting: The model hasn't learned enough patterns. Both training and validation loss remain high. The model lacks capacity to represent the data.

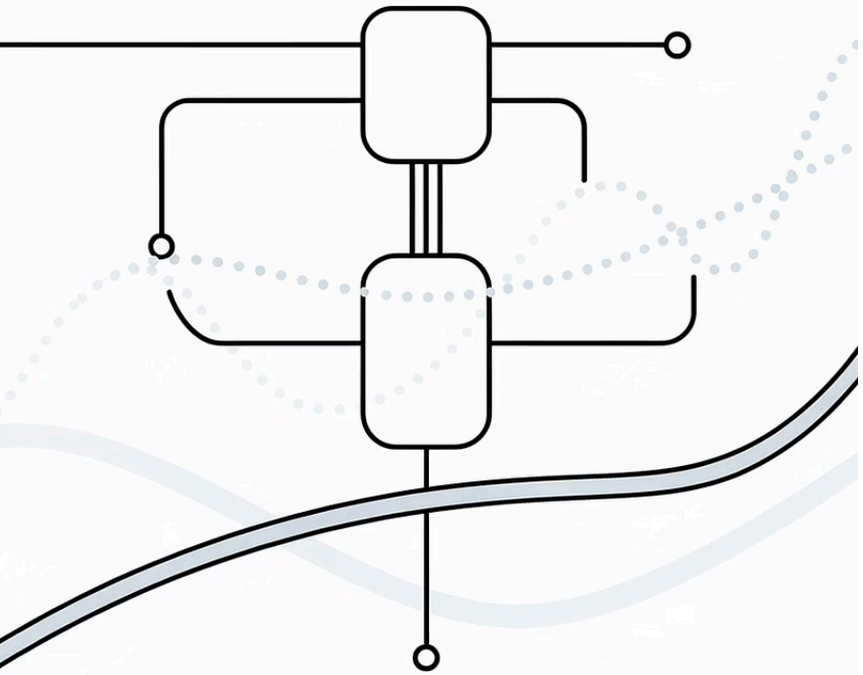
Just Right

Good Generalization: Training and validation loss are both low and close together. The model has learned the underlying patterns without memorizing noise.

Too Many Epochs

Overfitting: Training loss continues falling while validation loss rises. The model memorizes the training data and fails on unseen examples.

Early Stopping: Monitor validation loss and halt training when it starts increasing. PyTorch Lightning and Keras both support this natively.



Regularization: Keeping the Model Honest

Regularization techniques prevent overfitting by discouraging the model from learning overly complex, noise-specific patterns. They act as a form of **Occam's Razor** for neural networks.

L2 (Weight Decay)

Adds $\lambda \cdot ||W||^2$ to the loss. Penalizes large weights, pushing them toward zero. Equivalent to placing a Gaussian prior on weights. In PyTorch: `weight_decay` parameter in the optimizer.

L1 Regularization

Adds $\lambda \cdot |W|$ to the loss. Encourages **sparsity** — many weights become exactly zero. Useful for feature selection. Produces more interpretable models than L2.

Batch Normalization

Normalizes layer inputs to zero mean and unit variance per mini-batch. Stabilizes training, allows higher learning rates, and acts as an implicit regularizer. PyTorch: `nn.BatchNorm1d()`

Dropout: Regularization Through Randomness

Intuition

During each forward pass in training, each neuron is **randomly set to zero** with probability p (typically 0.2–0.5). This forces the network to not rely on any single neuron — every part of the network must be capable of working independently.

At test time, dropout is **disabled** and all neurons are active (outputs are scaled by (1−p) to compensate for the larger active set).

PyTorch Implementation

`nn.Dropout(p=0.5)` is placed between layers:

```
model = nn.Sequential(
  nn.Linear(128, 64),
  nn.ReLU(),
  nn.Dropout(p=0.5),
  nn.Linear(64, 10)
)
model.train()  # dropout active
model.eval()   # dropout disabled
```

Always call `model.eval()` before inference!

Autograd: Automatic Differentiation in PyTorch

PyTorch's **autograd** engine automatically computes gradients for all tensor operations that have `requires_grad=True`. It builds a **dynamic computational graph** during the forward pass and traverses it in reverse during `.backward()`.

How It Works

- Every operation on a tracked tensor records a **grad_fn** (gradient function).
- Calling `loss.backward()` triggers reverse-mode automatic differentiation through the graph.
- Gradients accumulate in `tensor.grad` attribute.

Code Example

```
x = torch.tensor(2.0,
    requires_grad=True)
y = x ** 2 + 3 * x

y.backward()
print(x.grad)  # dy/dx = 2x+3 = 7
```

Use `torch.no_grad()` context to disable tracking during inference, saving memory and compute.

Tensors & Batches: The Language of PyTorch

Everything in PyTorch is a **tensor** — a multi-dimensional array that lives on CPU or GPU. Training in **batches** allows efficient parallelization and provides gradient estimates with lower variance than single-sample updates.

Tensor Shapes

Input batch: `(B, D)` — B samples, D features. After hidden layer: `(B, H)`. Weight matrix W^1: `(H, D)`. Gradients match parameter shapes exactly.

Batch Processing

Mini-batch gradient descent balances noise and efficiency. Typical batch sizes: 32, 64, 128, 256. Larger batches → smoother gradients but more memory and potentially worse generalization.

DataLoader in PyTorch

`DataLoader(dataset, batch_size=64, shuffle=True)` handles batching, shuffling, and parallel data loading automatically. Shuffling prevents the model from learning the data order.

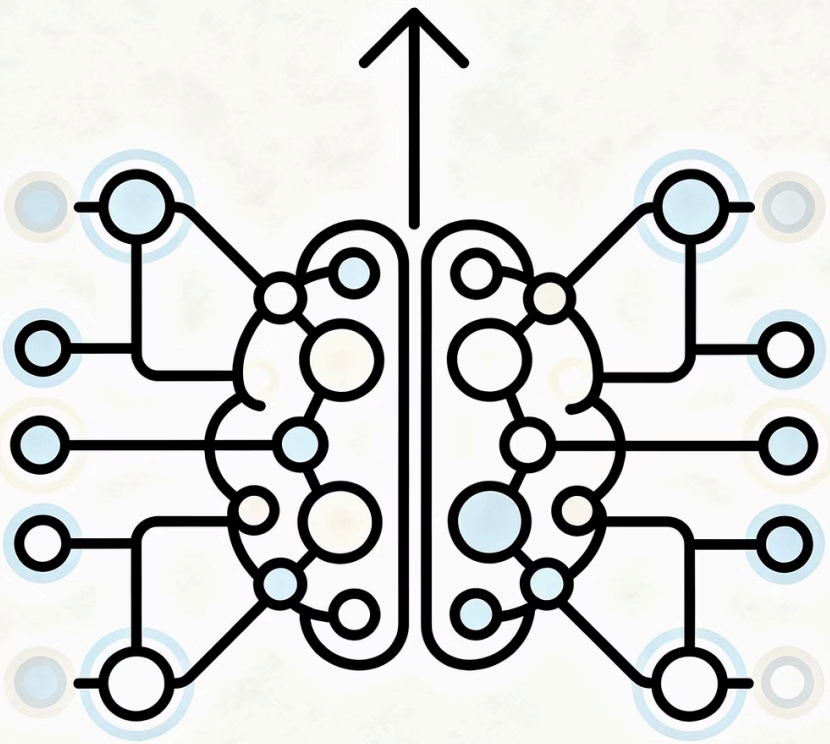
Full PyTorch: Feedforward Network End-to-End

```
import torch, torch.nn as nn

# Model Definition
class Net(nn.Module):
 def __init__(self):
 super().__init__()
 self.net = nn.Sequential(
 nn.Linear(784, 128),
 nn.ReLU(),
 nn.Dropout(0.3),
 nn.Linear(128, 10)
 )
 def forward(self, x):
 return self.net(x)

model = Net()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

# Training Loop
for epoch in range(20):
 model.train()
 for X, y in train_loader:
 pred = model(X)
 loss = criterion(pred, y)
 optimizer.zero_grad()
 loss.backward() # autograd magic!
 optimizer.step()
```



Key Takeaways

Backpropagation = Chain Rule

Gradients flow from loss back through every layer using the chain rule. Each layer receives a gradient and passes one further back.

Choose Loss & Optimizer Wisely

MSE for regression, Cross-Entropy for classification. Adam is a safe default optimizer; SGD+Momentum can outperform it with tuning.

Regularization Prevents Overfitting

Use Dropout, L2 weight decay, and early stopping together. Monitor both training and validation loss every epoch.

PyTorch Autograd Does the Heavy Lifting

Define your model with `nn.Module`, compute the loss, call `.backward()` — PyTorch handles all gradient computation automatically through the dynamic computation graph.